EE 250L: Distributed Systems for the Internet of Things

Fall 2026 - Lab 3

Web Services, HTTP, and RESTful APIs

Assigned: __15.sep.2026____ Submission due: ___21 || 23.sep.2026___

Lab outcome: Explain how HTTP and REST build on the transport concepts from Lab 2; build and test a small RESTful message service with Flask and JSON; consume WeatherAPI from Python; and make the same kind of HTTPS/JSON request from an ESP32-S3 using a heavily scaffolded ESP-IDF client.

How this lab extends Lab 2

Lab 2 used raw TCP sockets. Lab 3 moves up the stack to HTTP, REST, and JSON.

You will build and test a small REST API on your computer, then call a production API from Python and the ESP32-S3.

Part	Systems used	Main goal
1	Your computer's Unix command line (WSL, macOS Terminal, native Linux, or Lab 1 Ubuntu VM)	Connect TCP to HTTP/REST; complete and test a Flask message API with curl and a Python client.
2	Your computer's Unix command line + ESP32-S3	Use WeatherAPI from Python, then make a comparable HTTPS GET request from the ESP32-S3 and parse selected JSON fields.

The starter code contains:

File / directory	Purpose
mail_server.py	Flask REST API for storing and managing simple message resources.
mail_client.py	Python client used to exercise the required message endpoints.
weather.py	Python client for WeatherAPI.
esp32_weather_client/	ESP-IDF project for the ESP32-S3 WeatherAPI client; Wi-Fi, TLS, buffering, and HTTP-client plumbing are supplied.

Starter code: Complete only the marked TODOs; do not rebuild the supplied networking/TLS plumbing from scratch.

Part 1: Building and Testing a RESTful Service

1. Connect Lab 2 to HTTP and REST

TCP reliably moves bytes; HTTP defines request/response semantics on top of that transport. REST organizes HTTP APIs around resources, and JSON carries structured data in this lab.

Layer / idea	Example in this lab	What it provides
TCP	Underlying transport for the HTTP/1.1 requests used here	Reliable, ordered byte delivery between endpoints.
HTTP	GET /messages, POST /messages	Standard application request/response semantics, methods, headers, status codes, and bodies.
REST	Messages exposed as resources through URLs	Resource-oriented conventions for using HTTP methods and URLs.
JSON	{"sender":"alice", ...}	A text representation for structured application data.

Important distinction: HTTP and REST are related but not interchangeable. HTTP is a protocol. REST is an architectural style that is commonly implemented using HTTP.

Focus on five HTTP elements: method, URL/path, status code, headers, and body. JSON is used for request/response bodies.

2. Prepare Python, curl, and a virtual environment

Use WSL, macOS Terminal, or the Lab 1 Ubuntu VM. Create a Python virtual environment, so Flask and Requests are installed locally for this lab.

Windows: Use either WSL or the Ubuntu VM from Lab 1. WSL is not required.

- **Ubuntu / WSL / Debian-based Linux**

Install the required packages if needed:

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip curl
```

Create the Lab 3 directory and virtual environment:

```
mkdir -p ~/ee250/lab3
cd ~/ee250/lab3
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install --upgrade pip
python3 -m pip install flask requests
```

- **macOS - Terminal**

Open Terminal. Verify Python 3 and curl first:

```
python3 --version
curl --version
```

If Python 3 is unavailable, install the Xcode Command Line Tools when prompted. If you already use Homebrew, this is also fine:

```
brew install python
```

Then create the Lab 3 directory and virtual environment:

```
mkdir -p ~/ee250/lab3
cd ~/ee250/lab3
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install --upgrade pip
python3 -m pip install flask requests
```

Virtual environment: Activate .venv in each host-side terminal.
The ESP-IDF terminal used later is separate.

3. Obtain the Lab 3 starter files

Place the provided Python starter files in ~/ee250/lab3:

```
cd ~/ee250/lab3
 # Place mail_server.py, mail_client.py, and weather.py here
```

4. Understand the message resource model

The Flask server stores messages in memory only; restarting it clears them.

HTTP method	Path	Purpose	Success status
POST	/messages	Create a new message from a JSON body.	201 Created
GET	/messages	List all stored messages.	200 OK
GET	/messages?receiver=<name>	List messages addressed to one receiver.	200 OK
DELETE	/messages/<id>	Delete one stored message by identifier.	200 OK

The JSON object used to create a message has three required fields:

```
{
   "sender": "alice",
   "receiver": "bob",
   "message": "Hello from Lab 3"
 }
```

Code	Meaning	When you might see it here
200 OK	Request succeeded.	Successful GET or DELETE.
201 Created	A new resource was created.	Successful POST /messages.

400 Bad Request	The request data is invalid or incomplete.	Malformed JSON or a missing required field.
404 Not Found	The requested resource does not exist.	DELETE using an unknown message ID.

5. Complete mail_server.py

Open mail_server.py and complete only the marked TODOs.

Your completed server must:

- accept a JSON POST request containing sender, receiver, and message;
- reject a POST that is missing a required field with 400 Bad Request;
- store a valid message and return 201 Created;
- return all stored messages with GET /messages;
- support receiver filtering through GET /messages?receiver=<name>;
- delete a known message using DELETE /messages/<id> and return 200 OK with the confirmation format defined by the starter;
- return 404 Not Found when DELETE names an identifier that is not stored;
- use JSON responses consistently.

6. Start the completed Flask server and inspect HTTP with curl

After completing mail_server.py, start Flask on the supplied port 8000.

The starter code already sets port 8000 explicitly.

In Terminal 1:

```
cd ~/ee250/lab3
 source .venv/bin/activate
 python3 mail_server.py
```

In Terminal 2, activate the same .venv and issue a GET:

```
cd ~/ee250/lab3
 source .venv/bin/activate
 curl -i http://127.0.0.1:8000/messages
```

With -i, identify the status line, Content-Type header, and JSON body.

Now create a message with POST:

```
curl -i -X POST http://127.0.0.1:8000/messages \
   -H 'Content-Type: application/json' \
   -d '{"sender":"alice","receiver":"bob","message":"Hello from
curl"}'
```

GET /messages again, confirm the message appears, and note its id for DELETE.

Why curl? It provides an independent test outside the Python application. With -i, it displays the HTTP status line, response headers, and body directly.

7. Complete mail_client.py

Complete the mail_client.py TODOs.

Keep the Flask server running in one terminal, and use a second terminal for the client. Run one command at a time; each command sends one HTTP request.

Code helper

```
requests.get(url, params=params, timeout=10)
requests.post(url, json=payload, timeout=10)
requests.delete(url, timeout=10)
response.status_code
response.json()
```

At minimum, the client should:

1. Use the send command to POST at least two different messages.
2. Use the list command to GET the complete message collection.
3. Use list --receiver <name> to GET messages filtered by one receiver from your test messages.
4. Use the delete command with an identifier returned by the server.
5. Run list again and confirm the deleted message is gone.
6. For every command, print the HTTP status code and a readable form of the returned JSON.

Test an error case

Trigger at least one 4xx response, such as deleting an unknown id or sending an incomplete POST.

Questions for Part 1

1. In your own words, what structure does HTTP add on top of the raw TCP byte stream used in Lab 2?

2. What is the resource in this API? For POST /messages, GET /messages, GET /messages?receiver=<name>, and DELETE /messages/<id>, explain what each method/path combination does.

3. Name one common REST-related HTTP method that this message API does not use (for example PUT or PATCH). How could the API be extended to use it meaningfully?

Part 1 complete: Your Flask server and Python client implement the required message operations, and you can identify the important parts of the HTTP requests and responses.

Demo:

(a) one curl request with response headers visible,

(b) the Python client creating, listing/filtering, and deleting messages,

(c) at least one intentional 4xx response.

Part 2: Using an Existing RESTful API from Python and ESP32-S3

In Part 2, WeatherAPI operates the server. You will call it from Python and then from a scaffolded ESP32-S3 client.

1. Create a WeatherAPI account and API key

Create a free WeatherAPI account and locate your API key.

WeatherAPI: https://www.weatherapi.com/

Documentation: https://www.weatherapi.com/docs/

2. Complete weather.py

Activate .venv, place your API key in the starter placeholder, and complete weather.py. Ask for a city, send the GET request, check the status, and parse the requested JSON fields. Remove or obscure the real key before submission.

```
cd ~/ee250/lab3
 source .venv/bin/activate
 python3 weather.py
```

For a successful request, display at least:

- HTTP status code;
- resolved location name and region/country;
- temperature in degrees Fahrenheit;
- weather condition text;
- relative humidity.

After sending the request, check `response.status_code`. If the request fails, print the status code and the returned JSON error, then stop. Only parse the weather fields when the response is successful.

Run weather.py successfully for Los Angeles and for one other city of your choice.

3. Understand what the ESP32 starter code supplies

The ESP32 starter already handles Wi-Fi, TLS, and HTTP response buffering. Your TODOs stay at the HTTP/API/JSON level.

The supplied ESP-IDF project will already include:

- **Wi-Fi setup:** connection logic reused from the earlier ESP32 labs.
- **HTTPS/TLS configuration:** certificate verification through the ESP x509 certificate bundle; certificate verification must remain enabled.
- **HTTP response buffering:** the response body is accumulated correctly even when esp_http_client delivers data in multiple chunks.
- **cJSON:** the JSON parser used to extract fields from the completed response body.
- Task/buffer sizing: the supplied task stack and static HTTP response buffer are sized for the HTTPS exchange so the response buffer does not consume the task stack.

- **Cleanup/error paths:** client cleanup and low-level error handling are supplied unless explicitly marked as a TODO.

Starter detail: HTTP data may arrive in multiple chunks; the supplied code combines them before cJSON parsing.

4. Complete the small ESP32-S3 TODOs

The student TODOs in the ESP32 project are limited to the application-level pieces of the WeatherAPI exercise.

There are only a few TODOs:

1. Provide or select the WeatherAPI city/query used by the request
2. Construct/finalize the WeatherAPI request URL or query values from the supplied configuration
3. Check/print the returned HTTP status as directed by the starter
4. Use cJSON to extract the same selected weather fields used by weather.py.

Do not rewrite the Wi-Fi event handling, TLS setup, certificate bundle, HTTP event handler, response accumulator, or client cleanup unless the starter explicitly marks that code as a TODO.

ESP-IDF HTTP Client documentation for ESP32-S3:

https://docs.espressif.com/projects/esp-idf/en/release-v6.0/esp32s3/api-reference/protocols/esp_http_client.html

5. Configure ESP32 credentials and weather query

Open main/credentials.h in the provided ESP32 starter project and replace the placeholder values for WIFI_SSID, WIFI_PASSWORD, WEATHER_API_KEY, and WEATHER_QUERY.

If the weather query contains spaces, encode each space as %20 (for example, Los%20Angeles).

6. Build, flash, and monitor the ESP32-S3 client

Windows

Copy or extract the provided esp32_weather_client starter folder into your Lab 3 workspace. Open the ESP-IDF terminal/environment configured in Lab 1, then cd into that project directory.

```
idf.py set-target esp32s3
 idf.py build
 idf.py -p <PORT> flash monitor
```

macOS / Linux

Copy or extract the provided esp32_weather_client starter folder into your Lab 3 workspace. Open the ESP-IDF terminal/environment configured in Lab 1, then cd into that project directory. cd

```
cd <path-to-lab3-esp32-project>
 idf.py set-target esp32s3
 idf.py build
 idf.py -p <PORT> flash monitor
```

Replace <PORT> with the serial port for your board if automatic detection is not sufficient. On macOS/Linux, ls /dev/cu.* can help identify the board port; on Windows, use the COM port shown by the ESP-IDF tools or Device Manager. If the project target is already esp32s3, set-target does not need to be repeated on every build.

A successful serial-monitor run should show:

- successful Wi-Fi connection and an assigned IPv4 address;
- the start of the WeatherAPI request;
- HTTP status 200 for a successful request;
- the location, temperature, condition, and humidity extracted from the JSON response.

8. Compare the Python and ESP32 clients

Concept	Python client	ESP32-S3 client
HTTP library	requests	esp_http_client
Request	HTTPS GET	HTTPS GET
Response format	JSON	JSON
JSON parser	Python response/json handling	cJSON
Output	Terminal	ESP-IDF serial monitor

Although the programming languages and libraries differ, both clients must follow the same WeatherAPI contract: endpoint, query parameters, HTTP semantics, and JSON field structure.

Questions for Part 2

4. Why do many production APIs require an API key? What can the provider control or measure with it?

5. Why should the client check the HTTP status code before assuming the response body has the successful WeatherAPI JSON structure?

6. The Python and ESP32 implementations use different languages and libraries. What remains the same at the HTTP/API level?

Part 2 complete: You can consume a production REST API from Python and from the ESP32-S3, interpret HTTP status codes, and extract selected values from JSON.

Demo:

(a) successful Python WeatherAPI output for Los Angeles and one other city, and

(b) the ESP32-S3 serial monitor showing Wi-Fi connection, HTTP 200, and parsed weather output.

Optional Challenge

This challenge is not required for credit. If you want to connect the two halves of the lab, have the ESP32-S3 POST its weather reading to the Flask message service from Part 1 using esp_http_client. This reuses the ESP32-to-computer networking ideas from Lab 2. The computer running Flask must be reachable from the ESP32-S3 on the local network. This is usually straightforward on macOS, native Linux, or a Lab 1 Ubuntu

VM configured for direct local-network reachability. WSL2 normally sits behind a NATed virtual interface, so inbound ESP32-to-WSL connections require additional Windows port-forwarding and firewall configuration that is outside the scope of this optional challenge.

Submission

Each student should submit the completed starter code and the README/PDF file via Brightspace.

Your README/PDF should contain:

- Answers to Questions 1-6.
- An explicit acknowledgment of all assistance received, including AI tools, people, and websites, with a brief description of what help was provided.

Before submission: Check screenshots and source files for API keys, Wi-Fi passwords, or other credentials.

Demo and interview checklist

During the demo/interview, be prepared to demonstrate the working system and explain the code you submitted. The TA/CP may ask you to use a different message, receiver, or city from the one shown in your screenshots.

- Start the Flask server and demonstrate create, list/filter, and delete behavior using mail_client.py.
- Show a meaningful HTTP success or error status code and explain it.
- Run weather.py for a city requested by the interviewer and explain where the printed values appear in the JSON response.
- Show the ESP32-S3 requesting weather data and printing the parsed results.

Rubric

Points	Check	Evidence
15 (demo)	Part 1: Flask REST service	Working POST, GET/list/filter, DELETE, status handling, curl inspection.
10 (demo)	Part 2: WeatherAPI clients	Python and ESP32 clients return the required weather fields.
5	Submission	Correct code and answers in README/PDF + acknowledgment and description of assistance
70	Interview	–
100	Total	

Troubleshooting

Q: python3 -m venv .venv fails in Ubuntu/WSL/Debian-based Linux.
A: Install the venv package with sudo apt update and sudo apt install -y python3-venv, then retry. Do not work around this by repeatedly installing course packages into the system Python.

Q: Flask or requests cannot be imported, even though I installed them.
A: Activate the Lab 3 .venv in that terminal: source ~/ee250/lab3/.venv/bin/activate.

Q: The Flask server works in one terminal, but the client says connection refused.
A: Confirm Flask is still running on port 8000 and the client uses http://127.0.0.1:8000.

Q: Why does this lab use port 8000 instead of Flask's common port 5000?
A: Port 8000 avoids common macOS conflicts with port 5000. Use 8000 consistently in Flask, curl, and the client.

Q: curl or requests receive 404 Not Found.
A: Check the path exactly: /messages, DELETE /messages/<id>, and ?receiver=<name>.

Q: POST receives 400 Bad Request.
A: Confirm Content-Type: application/json and include sender, receiver, and message.

Q: idf.py is not found on Windows.
A: Use the ESP-IDF terminal/environment from Lab 1, not a generic shell.

Q: idf.py is not found on macOS / Linux.
A: Open the ESP-IDF terminal/environment from Lab 1; a normal shell may not have idf.py.

Q: The ESP32-S3 cannot join Wi-Fi.
A: Check 2.4 GHz Wi-Fi, SSID/password, and hotspot settings. On iPhone, Maximize Compatibility may help.

Q: The ESP32 HTTPS request fails before an HTTP status code appears.
A: Check Wi-Fi, DNS, TLS, and connection logs. Do not disable the starter certificate verification.

Q: Python works but the ESP32 JSON parsing fails.
A: Verify HTTP 200 and a complete response body. Keep the supplied multi-chunk response accumulation before cJSON parsing.

Q: I am using WSL and want to do the ESP32-to-Flask Optional Challenge.
A: WSL2 uses NAT, so this optional challenge may require Windows port-forwarding/firewall setup. macOS, native Linux, or a directly reachable Ubuntu VM is simpler.